

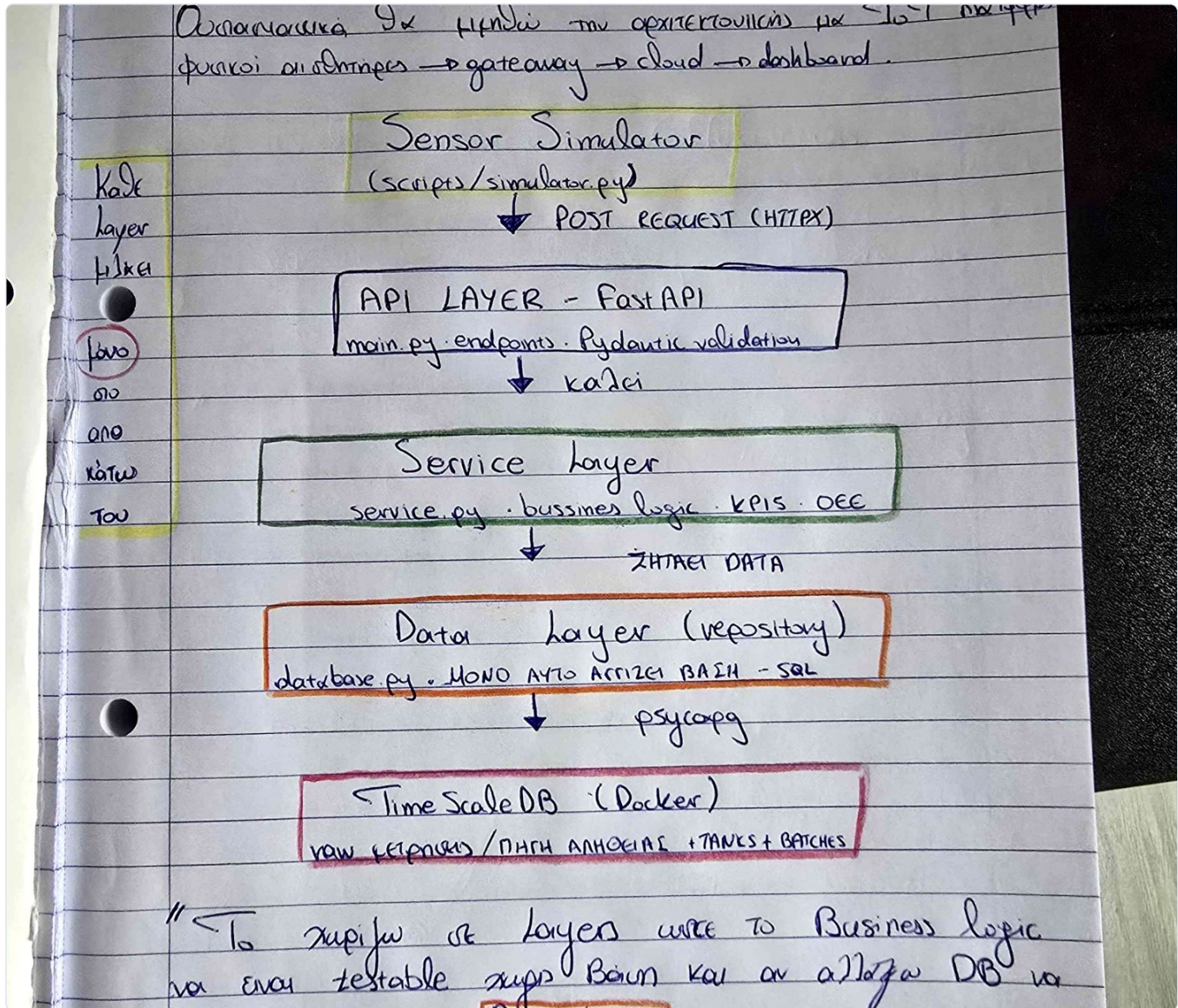
Brewery Digital Twin

Industrial IoT Backend — Python · FastAPI · TimescaleDB · Docker

Mini βιομηχανικό digital twin: αισθητήρες → ingestion → time-series βάση → analytics (KPIs / OEE) → live dashboard. Όλο το σύστημα σκηκώνεται με ένα `docker compose up`.

Αρχιτεκτονική

Layered design — κάθε επίπεδο μιλάει μόνο στο αμέσως από κάτω του. Η business logic μένει testable χωρίς βάση· αν αλλάξει η DB, πειράζεται μόνο το data layer.



Sensor Simulator → API (FastAPI) → Service → Data (repository) → TimescaleDB.

Μοντελοποίηση — Σταθερά / Μεταβλητά / Παράγωγα

Τρία είδη δεδομένων, καθένα στη σωστή του θέση: σταθερά → `tanks`, μεταβλητά (στιγμιαία) → `measurements`, παράγωγα (KPIs) → υπολογισμός στο service layer.

και κρατάμε και προ-υπολογισμένα aggregates για ταχύτητα
αλλά όχι ως ψηφή ΑΠΗΘΕΙΑΣ

Σταθερά vs

↓
Δεξαμενή : χωρητικότητα → tanks
όνομα

Μεταβλητά vs

↓
Σημεία : Θερμοκρασία → nivaras
πίεση measurements

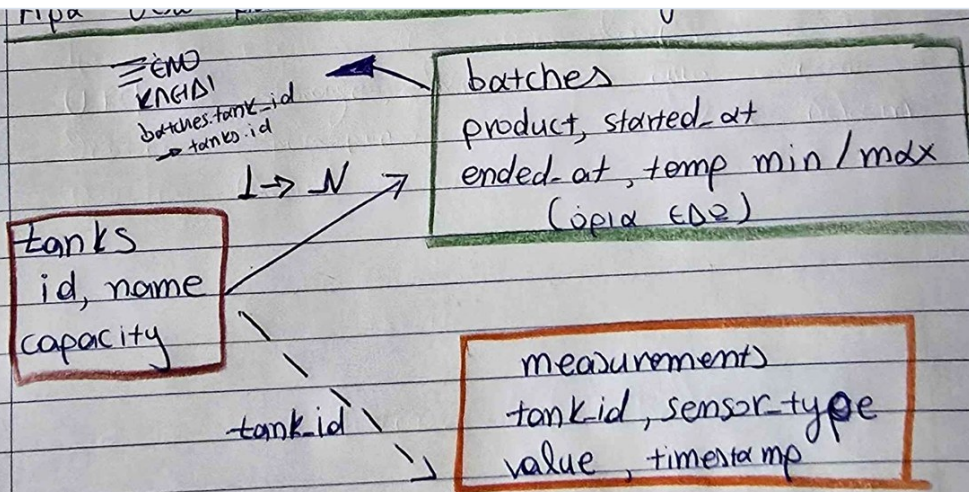
Παράγωγα vs

↓
KPIs : OEE, H.O., anomalies → υπολογισ στο
Service Layer

Σταθερά (δεξαμενή) · Μεταβλητά (θερμοκρασία/πίεση) · Παράγωγα (OEE, anomalies).

Data Model — Tanks & Batches

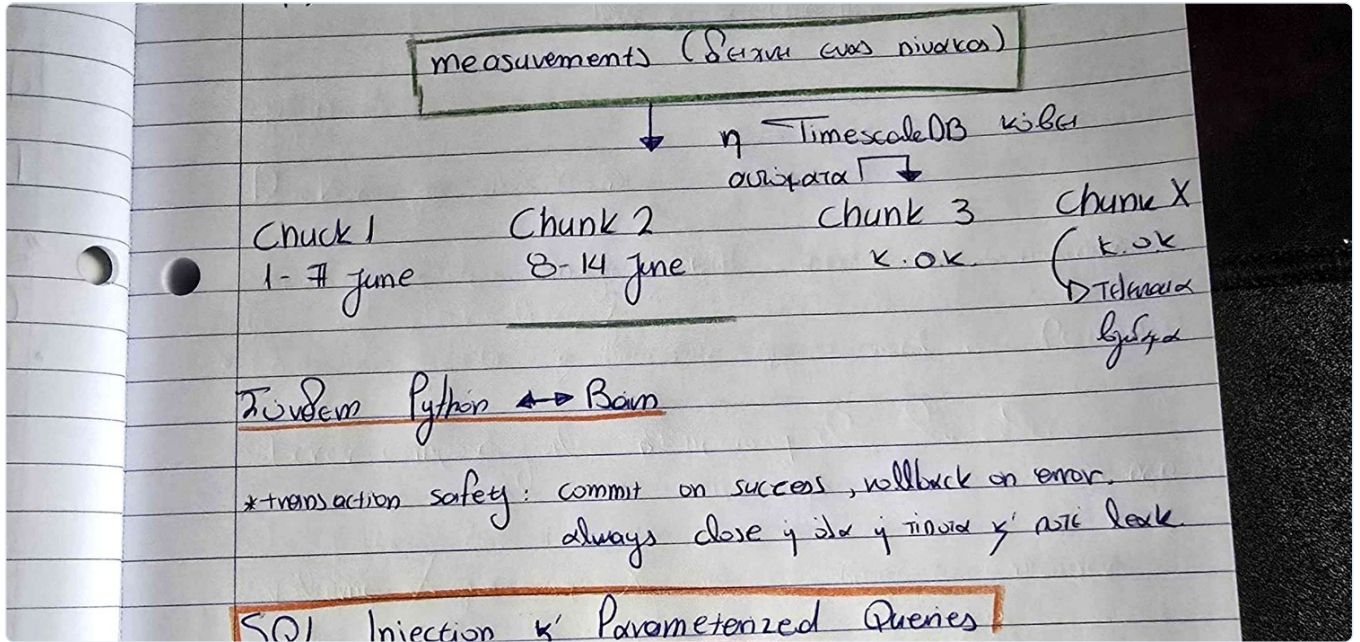
Τα όρια ανήκουν στο **batch**, όχι στο tank: η ίδια δεξαμενή βράζει διαφορετικά προϊόντα σε διαφορετικές περιόδους (1:N). FK batches.tank_id → tanks.id · το hypertable των measurements μένει χωρίς FK για γρήγορο ingestion.



ορισμένα σκεπτόμενα όρια ως ηδία των Tanks
αλλά ανα αλλάζω ανα προϊόν / περίοδο (1:N όχι 1:1)
οπότε επιβάλλεται χρονικά εξαρτημένο πορίδιο: batches.

tanks (σταθερά) · batches (όρια ανά προϊόν/περίοδο) · measurements (raw time-series).

To hypertable κόβει τον πίνακα σε chunks ανά χρόνο· τα queries κάνουν chunk exclusion και τρέχουν ακαριαία.



Διαμοιρασμός των measurements σε χρονικά chunks.

OEE = Availability × Performance × Quality, πολλαπλασιαστικό: ένας μηδενικός παράγοντας μηδενίζει ολόκληρο το OEE (ο μέσος όρος θα το έκρυβε).

Overall Equipment Effectiveness - OEE

Availability × Performance × Quality

Availability (δωλεψή) × Performance (χρησιμότητα) × Quality (καλώς; (1 - anomaly rate)) = OEE

▸ ΣΠΟΡΡΑΠΠΑΣΙΑΣΜΟΣ και όχι Μ.Ο.
 αν ένα ~~είναι~~ τότε στο OEE = ∅

πρακτικά ο μέσος όρος κρύβει μια καταστροφή στο
 πίσω αν το OEE είναι ∅.

π.χ. σε μέση η θερμοκρασία (2) μήτε στα 20°C ενώ εκεί
 όλο τους 14°C το OEE
 εκεί είναι ∅ γιατί το

Quality = 1 - anomaly rate · Availability = data continuity · Performance = τεκμηριωμένο placeholder.

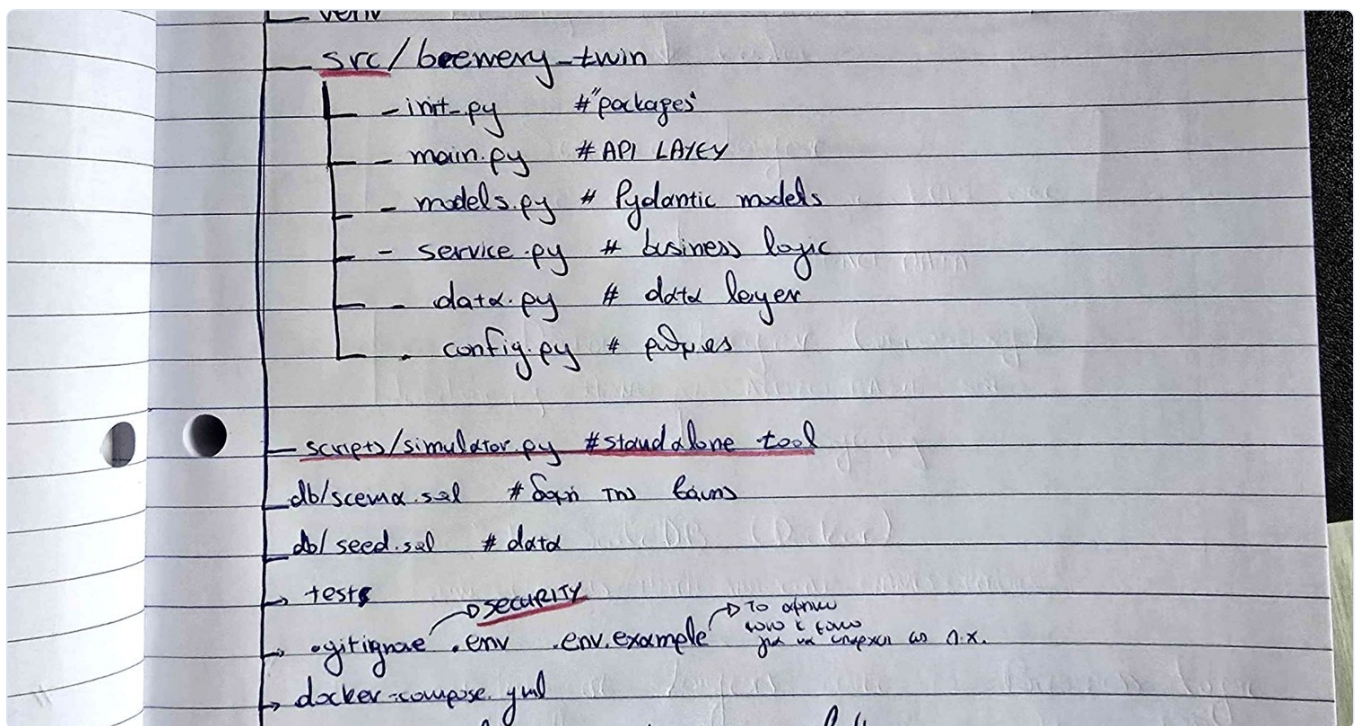
API Endpoints

Σύνοψη Endpoints & Req	π. κανονική	Layers
GET /health	ελέγχο jup	API
POST /measurements	αποθηκεύει δεδομένα	API → service → Data → DB
GET /tanks/{id}/stats	avg/min/max	"
GET /tanks/{id}/anomalies	επίσημο error όπως	"
GET /tanks/{id}/oee	OEE στο data sensor	API - service (anomalies) Data → DB

health · measurements (POST) · tanks/{id}/stats · /anomalies · /oee.

Δομή Project

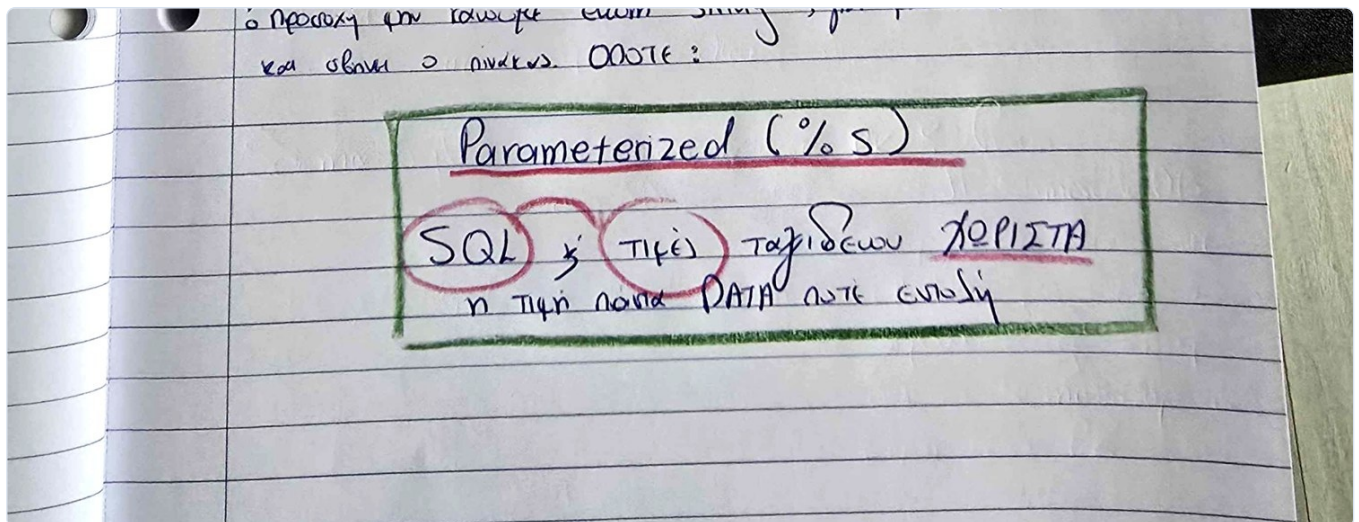
Καθαρός διαχωρισμός: application package (src/), standalone simulator (scripts/), schema as code (db/), tests, και secrets εκτός git.



src/brewery_twin (κώδικας) · scripts (tool) · db (schema/seed) · tests · .env (git-ignored).

Security — Parameterized Queries

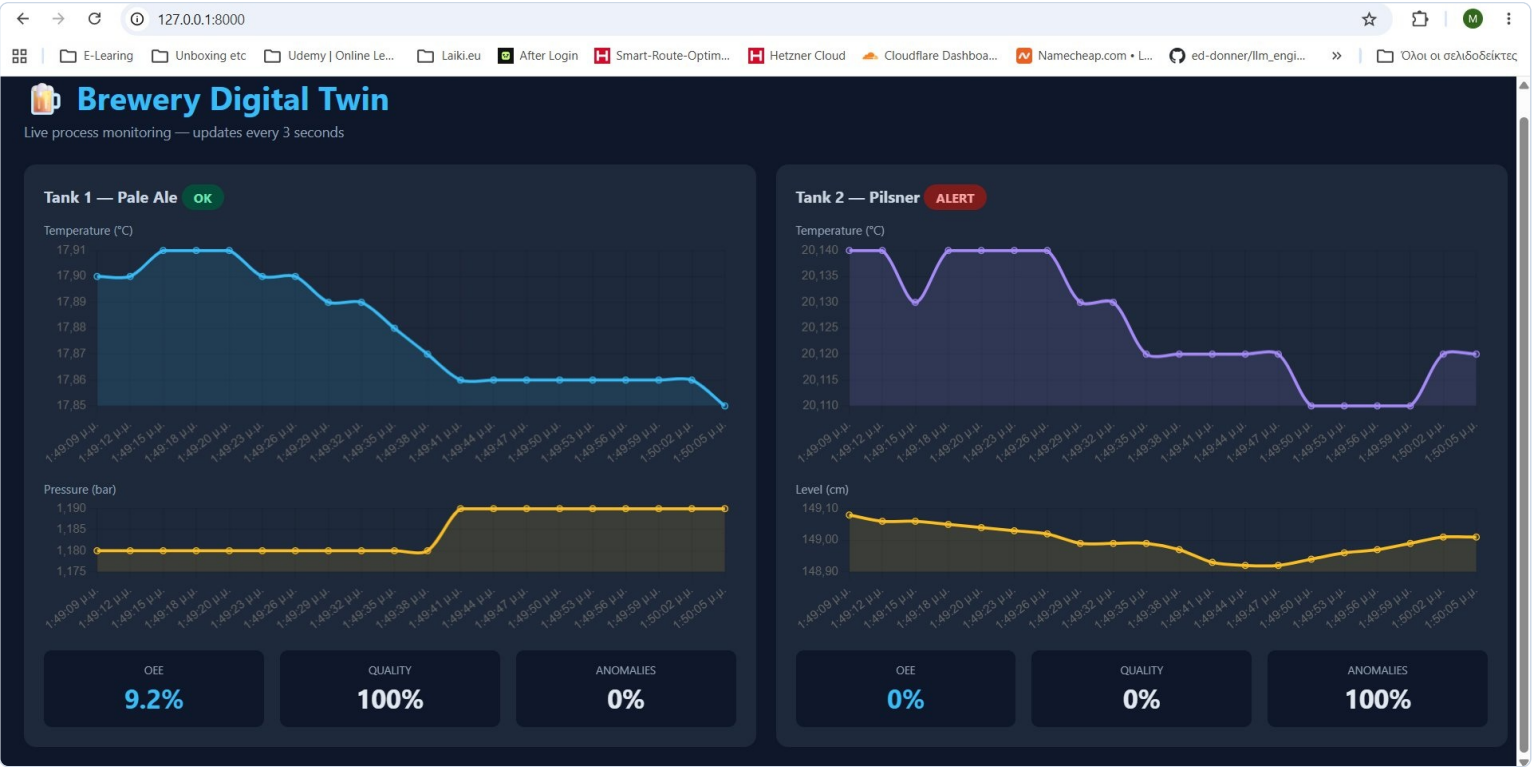
SQL και τιμές ταξιθεύουν χωριστά: η τιμή αντιμετωπίζεται πάντα ως δεδομένο, ποτέ ως εκτελέσιμη εντολή — αποτρέπει το SQL injection.



Parameterized (%s): η τιμή είναι πάντα DATA, ποτέ command.

Live Dashboard

Σερβίρεται από το ίδιο το API, ανανέωση κάθε 3s. Tank 1 (εντός ορίων) → OK · Tank 2 (Pilsner ~20°C ενώ όριο 14°C) → 100% anomalies, Quality 0%, OEE 0% → ALERT.



Δύο δεξαμενές, live γραφήματα (θερμοκρασία + πίεση/στάθμη), KPIs, status badge.

OEE — σταθεροποίηση με τον χρόνο

Το Availability εξαρτάται από το πλήθος μετρήσεων στο παράθυρο. Αμέσως μετά την εκκίνηση είναι χαμηλό και ανεβαίνει σταθερά καθώς συσσωρεύονται δεδομένα.



Λίγο μετά την εκκίνηση — Tank 1 OEE χαμηλό (το window δεν έχει γεμίσει).

Live process monitoring — updates every 3 seconds



Μετά από περισσότερο χρόνο — το OEE ανεβαίνει καθώς το availability window γεμίζει.